

N	$\pi(n)$
---	----------

Bài toán đặt ra được dẫn về bài toán tối ưu tổ hợp: $\min\{f(\pi) : \pi \in \Pi\}$.

Bài toán lập lịch. Mỗi một chi tiết trong số n chi tiết $D_1, D_2, \dots, D_n$ cần phải lần lượt được gia công trên m máy $M_1, M_2, \dots, M_m$. Thời gian gia công chi tiết D_i trên máy M_j là t_{ij} . Hãy tìm lịch (trình tự gia công) các chi tiết trên các máy sao cho việc hoàn thành gia công tất cả các chi tiết là sớm nhất có thể được. Biết rằng, các chi tiết được gia công một cách liên tục, nghĩa là quá trình gia công của mỗi một chi tiết phải được tiến hành một cách liên tục hết máy này sang máy khác không cho phép có khoảng thời gian dừng khi chuyển từ máy này sang máy khác.

Tập phương án của bài toán. Rõ ràng, mỗi một lịch gia công các chi tiết trên các máy sẽ tương ứng với một hoán vị $\pi = (\pi(1), \pi(2), \dots, \pi(n))$ của n số tự nhiên $1, 2, \dots, n$.

Hàm mục tiêu của bài toán. Thời gian hoàn thành theo các lịch trên được xác định bởi hàm số

$$f(\pi) = \sum_{j=1}^{n-1} C_{\pi(j), \pi(j+1)} + \sum_{k=1}^m t_{k, \pi(n)}, \text{ trong đó } c_{ij} = S_j - S_i, S_j \text{ là thời điểm bắt đầu thực}$$

hiện việc gia công chi tiết j ($i, j = 1, 2, \dots, n$). Ý nghĩa của hệ số c_{ij} có thể được giải thích như sau: nó là tổng thời gian gián đoạn (được tính từ khi bắt đầu gia công chi tiết i) gây ra bởi chi tiết j khi nó được gia công sau chi tiết i trong lịch gia công. Vì vậy, c_{ij} có thể tính theo công thức:

$$c_{ij} = \max_{1 \leq k \leq m} \left[\sum_{l=1}^k t_{lj} - \sum_{l=1}^{k-1} t_{li} \right], i, j = 1, 2, \dots, n. \text{ Vì vậy bài toán đặt ra dẫn về bài toán}$$

tối ưu tổ hợp sau:

$$\min \{ f(\pi) : \pi \in \Pi \}.$$

Trong thực tế, lịch gia công còn phải thỏa mãn thêm nhiều điều kiện khác nữa. Vì những ứng dụng quan trọng của những bài toán loại này mà trong tối ưu hoá tổ hợp đã hình thành một lĩnh vực lý thuyết riêng về các bài toán lập lịch gọi là lý thuyết lập lịch hay qui hoạch lịch.

4.2. Phương pháp duyệt toàn bộ

Một trong những phương pháp hiển nhiên có thể giải bài toán tối ưu tổ hợp là duyệt tất cả các phương án của bài toán. Ứng với mỗi phương án, ta đều tính được giá trị của hàm mục tiêu cho phương án đó, sau đó so sánh giá trị của hàm mục tiêu tại tất cả các phương án đã được liệt kê để tìm ra phương án tối ưu. Phương pháp xây dựng theo nguyên tắc như trên được gọi là phương pháp duyệt toàn bộ. Thuật toán tổng quát thực hiện theo phương pháp duyệt toàn bộ được thể hiện trong Hình 4.1.

Hạn chế của phương pháp duyệt toàn bộ là sự bùng nổ của các cấu hình tổ hợp. Chẳng hạn để duyệt được $15! = 1\,307\,674\,368\,000$ cấu hình, trên máy có tốc độ 1 tỷ phép

tính giây, nếu mỗi hoán vị cần liệt kê mất khoảng 100 phép tính, thì ta cần khoảng thời gian là 130767 giây (lớn hơn 36 tiếng đồng hồ). Vì vậy, cần phải có biện pháp hạn chế việc kiểm tra hoặc tìm kiếm trên các cấu hình tổ hợp thì mới có hy vọng giải được các bài toán tối ưu tổ hợp thực tế. Tất nhiên, để đưa ra được một thuật toán cần phải nghiên cứu kỹ tính chất của mỗi bài toán tổ hợp cụ thể. Chính nhờ những nghiên cứu đó, trong một số trường hợp cụ thể ta có thể xây dựng được thuật toán hiệu quả để giải quyết bài toán đặt ra. Tuy vậy, cũng cần phải chú ý rằng nhiều bài toán tối ưu hiện nay vẫn chưa tìm ra được một phương pháp hữu hiệu nào ngoài phương pháp duyệt toàn bộ đã được đề cập ở trên.

Thuật toán duyệt toàn bộ:

Bước 1 (Khởi tạo):

$XOPT = \emptyset$; //Khởi tạo phương án tối ưu ban đầu

$FOPT = -\infty (+\infty)$; //Khởi tạo giá trị tối ưu ban đầu

Bước 2 (Lặp):

for each $X \in D$ do { //lấy mỗi phần tử trên tập phương án

$S = f(X)$; // tính giá trị hàm mục tiêu cho phương án X

if ($FOPT < S$) { //Cập nhật phương án tối ưu

$FOPT = S$; //Giá trị tối ưu mới được xác lập

$XOPT = X$; // Phương án tối ưu mới

}

}

Bước 3 (Trả lại kết quả):

Return($XOPT, FOPT$);

Hình 4.1. Thuật toán duyệt toàn bộ.

Ghi chú. Tại Bước 1 của thuật toán, ta khởi tạo giá trị của hàm mục tiêu $FOPT = -\infty$ với bài toán tìm **max**, khởi tạo giá trị của hàm mục tiêu $FOPT = +\infty$ với bài toán tìm **min**.

Ví dụ 1. Giải bài toán cái túi dưới đây bằng phương pháp duyệt toàn bộ.

$$\begin{cases} F(X) = 4x_1 + 6x_2 + 3x_3 + 5x_4 + 2x_5 \rightarrow \max, \\ 9x_1 + 8x_2 + 5x_3 + 3x_4 + 2x_5 \leq 21, \\ x_j \in \{0,1\}, j = 1,2,3,4,5. \end{cases}$$

Lời giải. Gọi $A = (a_1, a_2, \dots, a_n)$, $C = (c_1, c_2, \dots, c_n)$ tương ứng với vector trọng lượng và giá trị sử dụng các đồ vật. Như đã phân tích trong Mục 4.1, ta có:

Tập các phương án của bài toán: $D = \left\{ X = (x_1, x_2, \dots, x_n) : \sum_{i=1}^n a_i x_i \leq b \right\}.$

Hàm mục tiêu của bài toán: $f(X) = \sum_{i=1}^n c_i x_i.$

Nhiệm vụ của chúng ta là tìm phương án tối ưu XOPT và giá trị tối ưu FOPT. Kết quả thực theo thuật toán duyệt toàn bộ được thể hiện theo Bảng 4.1 dưới đây.

$X=(x_1, x_2, x_3, x_4, x_5)$	$X \in D = \{x_1, x_2, x_3, x_4, x_5 : \sum_{i=1}^5 a_i x_i \leq 21\}$	$F(X) = \sum_{i=1}^5 c_i x_i = ?$
0, 0, 0, 0, 0	$0 \leq 21: X \in D$	0
0, 0, 0, 0, 1	$2 \leq 21: X \in D$	2
0, 0, 0, 1, 0	$3 \leq 21: X \in D$	5
0, 0, 0, 1, 1	$5 \leq 21: X \in D$	7
0, 0, 1, 0, 0	$5 \leq 21: X \in D$	3
0, 0, 1, 0, 1	$7 \leq 21: X \in D$	5
0, 0, 1, 1, 0	$8 \leq 21: X \in D$	8
0, 0, 1, 1, 1	$10 \leq 21: X \in D$	10
0, 1, 0, 0, 0	$8 \leq 21: X \in D$	6
0, 1, 0, 0, 1	$10 \leq 21: X \in D$	8
0, 1, 0, 1, 0	$11 \leq 21: X \in D$	11
0, 1, 0, 1, 1	$13 \leq 21: X \in D$	13
0, 1, 1, 0, 0	$13 \leq 21: X \in D$	9
0, 1, 1, 0, 1	$15 \leq 21: X \in D$	11
0, 1, 1, 1, 0	$16 \leq 21: X \in D$	14
0, 1, 1, 1, 1	$18 \leq 21: X \in D$	16
1, 0, 0, 0, 0	$9 \leq 21: X \in D$	4
1, 0, 0, 0, 1	$11 \leq 21: X \in D$	6
1, 0, 0, 1, 0	$12 \leq 21: X \in D$	9
1, 0, 0, 1, 1	$14 \leq 21: X \in D$	9
1, 0, 1, 0, 0	$14 \leq 21: X \in D$	10
1, 0, 1, 0, 1	$16 \leq 21: X \in D$	9
1, 0, 1, 1, 0	$16 \leq 21: X \in D$	12
1, 0, 1, 1, 1	$19 > 21: X \notin D$	14
1, 1, 0, 0, 0	$17 \leq 21: X \in D$	10
1, 1, 0, 0, 1	$19 \leq 21: X \in D$	12
1, 1, 0, 1, 0	$20 \leq 21: X \in D$	15
1, 1, 0, 1, 1	$22 > 21: X \notin D$	∞
1, 1, 1, 0, 0	$22 > 21: X \notin D$	∞
1, 1, 1, 0, 1	$24 > 21: X \notin D$	∞
1, 1, 1, 1, 0	$25 > 21: X \notin D$	∞

1, 1, 1, 1, 1	27>21: X $\notin$ D	∞
XOPT=(0,1,1,1,1); FOPT = 16		

4.3. Thuật toán nhánh cận

Giả sử chúng ta cần giải quyết bài toán tối ưu tổ hợp với mô hình tổng quát như sau:

$$\text{Tìm min } \{f(x) : x \in D\}.$$

Trong đó D là tập hữu hạn phân tử. Ta giả thiết D được mô tả như sau:

$D = \{x = (x_1, x_2, \dots, x_n) \in A_1 \times A_2 \times \dots \times A_n; x \text{ thỏa mãn tính chất } P\}$, với $A_1 \times A_2 \times \dots \times A_n$ là các tập hữu hạn, P là tính chất cho trên tích đề xác $A_1 \times A_2 \times \dots \times A_n$.

Như vậy, các bài toán chúng ta vừa trình bày ở trên đều có thể được mô tả dưới dạng trên. Với giả thiết về tập D như trên, chúng ta có thể sử dụng thuật toán quay lui để liệt kê các phương án của bài toán. Trong quá trình liệt kê theo thuật toán quay lui, ta sẽ xây dựng dần các thành phần của phương án. Ta gọi, một bộ phận gồm k thành phần $(a_1, a_2, \dots, a_k)$ xuất hiện trong quá trình thực hiện thuật toán sẽ được gọi là phương án bộ phận cấp k .

Thuật toán nhánh cận có thể được áp dụng giải bài toán đặt ra nếu như có thể tìm được một hàm g xác định trên tập tất cả các phương án bộ phận của bài toán thỏa mãn bất đẳng thức sau:

$$g(a_1, a_2, \dots, a_k) \leq \min \{f(x) : x \in D, x_i = a_i, i = 1, 2, \dots, k\} \quad (*)$$

với mọi lời giải bộ phận $(a_1, a_2, \dots, a_k)$, và với mọi $k = 1, 2, \dots$.

Bất đẳng thức $(*)$ có nghĩa là giá trị của hàm tại phương án bộ phận $(a_1, a_2, \dots, a_k)$ không vượt quá giá trị nhỏ nhất của hàm mục tiêu bài toán trên tập con các phương án.

$$D(a_1, a_2, \dots, a_k) = \{x \in D : x_i = a_i, i = 1, 2, \dots, k\},$$

nói cách khác, $g(a_1, a_2, \dots, a_k)$ là cận dưới của tập $D(a_1, a_2, \dots, a_k)$. Do có thể đồng nhất tập $D(a_1, a_2, \dots, a_k)$ với phương án bộ phận $(a_1, a_2, \dots, a_k)$, nên ta cũng gọi giá trị $g(a_1, a_2, \dots, a_k)$ là cận dưới của phương án bộ phận $(a_1, a_2, \dots, a_k)$.

Giả sử ta đã có được hàm g . Ta xét cách sử dụng hàm này để hạn chế khối lượng duyệt trong quá trình duyệt tất cả các phương án theo thuật toán quay lui. Trong quá trình liệt kê các phương án có thể đã thu được một số phương án của bài toán. Gọi $\bar{x}$ là giá trị hàm mục tiêu nhỏ nhất trong số các phương án đã duyệt, ký hiệu $\bar{f} = f(\bar{x})$. Ta gọi $\bar{x}$ là phương án tốt nhất hiện có, còn $\bar{f}$ là kỷ lục. Giả sử ta có được $\bar{f}$, khi đó nếu

$$g(a_1, a_2, \dots, a_k) > \bar{f} \text{ thì từ bất đẳng thức } (*) \text{ ta suy ra}$$

$\bar{f} < g(a_1, a_2, \dots, a_k) \leq \min \{f(x) : x \in D, x_i = a_i, i = 1, 2, \dots, k\}$, vì thế tập con các phương án của bài toán $D(a_1, a_2, \dots, a_k)$ chắc chắn không chứa phương án tối ưu. Trong

trường hợp này ta không cần phải phát triển phương án bộ phận $(a_1, a_2, \dots, a_k)$, nói cách khác là ta có thể loại bỏ các phương án trong tập $D(a_1, a_2, \dots, a_n)$ khỏi quá trình tìm kiếm.

Thuật toán quay lui liệt kê các phương án cần sửa đổi lại như Hình 4.2:

```

Procedure Try( k ){
  /*Phát triển phương án bộ phận  $(a_1, a_2, \dots, a_{k-1})$  theo thuật toán quay
  lui có kiểm tra cận dưới Trước khi tiếp tục phát triển phương án*/
  for (  $a_k \in A_k$  ) {
    if ( chấp nhận  $a_k$  ){
       $x_k = a_k$ ;
      if (  $k == n$  )
        < cập nhật kỷ lục>;
      else if (  $g(a_1, a_2, \dots, a_k) \leq \bar{f}$  )
        Try (  $k+1$  );
    }
  }
}

```

Hình 4.2. Thuật toán quay lui dựa vào hàm đánh giá cận.

Khi đó, thuật toán nhánh cận được thực hiện như Hình 4.3.

```

Procedure Nhanh_Can( ) {
   $\bar{f} = +\infty$ ; /* Nếu biết một phương án  $\bar{x}$  nào đó thì có thể đặt  $\bar{f} = f(\bar{x})$ . */
  Try(1); //Thực hiện thuật toán quay lui trong Hình 4.2
  if (  $\bar{f} \leq +\infty$  )
    <  $\bar{f}$  là giá trị tối ưu ,  $\bar{x}$  là phương án tối ưu >;
  else
    < bài toán không có phương án>;
}

```

Hình 4.3. Thuật toán nhánh cận.

Chú ý rằng nếu trong thủ tục Try ta thay thế câu lệnh

if ($k == n$) < cập nhật kỷ lục >;
 else if ($g(a_1, a_2, \dots, a_k) \leq \bar{f}$) Try($k+1$);

bởi

if ($k == n$)
 < cập nhật kỷ lục >;
 else Try($k+1$);

thì thủ tục Try sẽ liệt kê toàn bộ các phương án của bài toán. Việc xây dựng hàm g phụ thuộc vào từng bài toán tối ưu tổ hợp cụ thể. Nhưng chúng ta cố gắng xây dựng sao cho đạt được những điều kiện dưới đây:

- ☐ Việc tính giá trị của g phải đơn giản hơn việc giải bài toán tổ hợp trong vế phải của (*).
- ☐ Giá trị của $g(a_1, a_2, \dots, a_k)$ phải sát với giá trị vế phải của (*).

Rất tiếc, hai yêu cầu này trong thực tế thường đối lập nhau.

Ví dụ 1. Bài toán cái túi. Chúng ta sẽ xét bài toán cái túi tổng quát hơn mô hình đã được trình bày trong mục 4.1. Thay vì có n đồ vật, ở đây ta giả thiết rằng có n loại đồ vật và số lượng đồ vật mỗi loại là không hạn chế. Khi đó, ta có mô hình bài toán cái túi biến nguyên sau đây: Có n loại đồ vật, đồ vật thứ j có trọng lượng a_j và giá trị sử dụng c_j ($j = 1, 2, \dots, n$). Cần chất các đồ vật này vào một cái túi có trọng lượng là b sao cho tổng giá trị sử dụng của các đồ vật đựng trong túi là lớn nhất.

Mô hình toán học của bài toán có dạng sau tìm

$$f^* = \max \left\{ f(x) = \sum_{j=1}^n c_j x_j : \sum_{j=1}^n a_j x_j \leq b, x_j \in Z_+, j = 1, 2, \dots, n \right\}, (1).$$

Trong đó Z^+ là tập các số nguyên không âm.

Ký hiệu D là tập các phương án của bài toán (1):

$$D = \left\{ x = (x_1, x_2, \dots, x_n) : \sum_{j=1}^n a_j x_j \leq b, x_j \in Z_+, j = 1, 2, \dots, n \right\}.$$

Không giảm tính tổng quát ta giả thiết rằng, các đồ vật được đánh số sao cho bất đẳng thức sau được thoả mãn

$$\frac{c_1}{a_1} \geq \frac{c_2}{a_2} \geq \dots \geq \frac{c_n}{a_n} \quad (2)$$

Để xây dựng hàm tính cận dưới, cùng với bài toán cái túi (1) ta xét bài toán cái túi biến liên tục sau: Tìm

$$g^* = \max \left\{ \sum_{j=1}^n c_j x_j : \sum_{j=1}^n a_j x_j \leq b, x_j \geq 0, j = 1, 2, \dots, n \right\}. \quad (3)$$

Mệnh đề. Phương án tối ưu của bài toán (3) là vector $\bar{x} = (\bar{x}_1, \bar{x}_2, \dots, \bar{x}_n)$ với các thành phần được xác định bởi công thức:

$$\bar{x}_1 = \frac{b}{a_1}, \bar{x}_2 = \bar{x}_3 = \dots = \bar{x}_n = 0 \text{ và giá trị tối ưu là } g^* = \frac{c_1 b_1}{a_1}.$$

Chứng minh. Thực vậy, xét $x = (x_1, x_2, \dots, x_n)$ là một phương án tùy ý của bài toán (3). Khi đó từ bất đẳng thức (3) và do $x_j \geq 0$, ta suy ra

$$c_j x_j \geq (c_1 / a_1) a_j x_j, j = 1, 2, \dots, n.$$

suy ra:

$$\sum_{j=1}^n c_j x_j \leq \sum_{j=1}^n \left(\frac{c_1}{a_1} \right) a_j x_j = \left(\frac{c_1}{a_1} \right) \sum_{j=1}^n a_j x_j \leq \frac{c_1}{a_1} b = g^*. \text{ Mệnh đề được chứng minh.}$$

Bây giờ ta giả sử có phương án bộ phận cấp k : $(u_1, u_2, \dots, u_k)$. Khi đó giá trị sử dụng của các đồ vật đang có trong túi là

$$\partial_k = c_1 u_1 + c_2 u_2 + \dots + c_k u_k, \text{ và trọng lượng còn lại của túi là}$$

$$b_k = b - c_1 u_1 + c_2 u_2 + \dots + c_k u_k,$$

ta có

$$\begin{aligned} & \max \{ f(x) : x \in D, x_j = u_j, j = 1, 2, \dots, n \} \\ &= \max \left\{ \partial_k + \sum_{j=k+1}^n c_j x_j : \sum_{j=k+1}^n a_j x_j \leq b_k, x_j \in Z_+, j = k+1, k+2, \dots, n \right\} \\ &\leq \partial_k + \max \left\{ \sum_{j=k+1}^n c_j x_j : \sum_{j=k+1}^n a_j x_j \leq b_k, x_j \geq 0, j = k+1, k+2, \dots, n \right\} \\ &= \partial_k + \frac{c_{k+1} b_k}{a_{k+1}} \end{aligned}$$

$$(\text{Theo mệnh đề giá trị số hạng thứ hai là } \frac{c_{k+1} b_k}{a_{k+1}})$$

Vậy ta có thể tính cận trên cho phương án bộ phận $(u_1, u_2, \dots, u_k)$ theo công thức

$$g(u_1, u_2, \dots, u_k) = \partial_k + \frac{c_{k+1} b_k}{a_{k+1}} \dots$$

Chú ý: Khi tiếp tục xây dựng thành phần thứ $k+1$ của lời giải, các giá trị đề cử cho x_{k+1} sẽ là $0, 1, \dots, [b_k / a_{k+1}]$. Do có kết quả của mệnh đề, khi chọn giá trị cho x_{k+1} ta sẽ duyệt

các giá trị đề cử theo thứ tự giảm dần. Thuật toán nhánh cận giải bài toán cái túi được mô tả như Hình 4.4 dưới đây.

Thuật toán Brach_And_Bound (i) {

```

     $t = (b - b_i) / A[i];$  // Khởi tạo số lượng đồ vật thứ  $i$ 
    for  $j = t; j \geq 0; j--$  {
         $x[i] = j;$  // Lựa chọn  $x[i]$  là  $j$ ;
         $b_i = b_i + a_i x_i;$  // Trọng lượng túi cho bài toán bộ phận thứ  $i$ .
         $\sigma_i = \sigma_i + c_i x_i;$  // Giá trị sử dụng cho bài toán bộ phận thứ  $i$ 
        If  $(k = n) < \text{Cập nhật kỷ lục}>;$ 
        else if  $(\delta_k + (c_{k+1} * b_k) / a_{k+1} > FOPT)$  // Nhánh cận được triển khai tiếp theo
            Branch_And_Bound( $k+1$ );
         $b_i = b_i - a_i x_i;$ 
         $\sigma_i = \sigma_i - c_i x_i;$ 
    }
}
```

Hình 4.4. Thuật toán nhánh cận giải bài toán cái túi

Ví dụ. Giải bài toán cái túi sau theo thuật toán nhánh cận trình bày ở trên.

$$f(x) = 10x_1 + 5x_2 + 3x_3 + 6x_4 \rightarrow \max$$

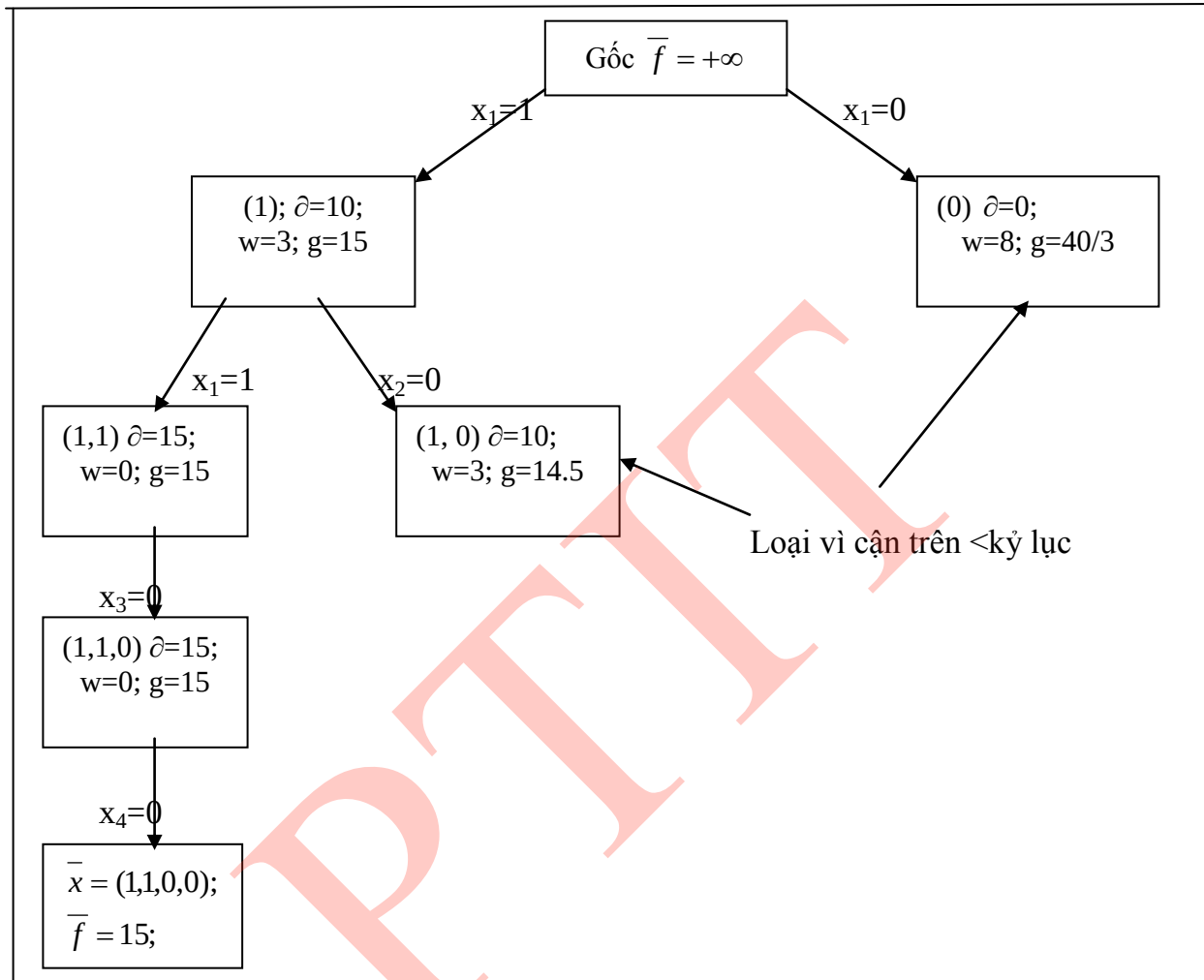
$$5x_1 + 3x_2 + 2x_3 + 4x_4 \leq 8$$

$$x_j \in Z_+, j = 1, 2, 3, 4.$$

Lời giải. Quá trình giải bài toán được mô tả trong cây tìm kiếm trong Hình 4.4. Thông tin về một phương án bộ phận trên cây được ghi trong các ô trên hình vẽ tương ứng theo thứ tự sau:

- Đầu tiên là các thành phần của phương án bộ phận thứ i : $X = (x_1, x_2, \dots, x_i)$;
- Tiếp đến θ là giá trị của các đồ vật theo phương án bộ phận thứ i ;
- Kế tiếp là w tương ứng với trọng lượng còn lại của túi;
- Cuối cùng là cận trên g .

Kết thúc thuật toán, ta thu được phương án tối ưu là $x^* = (1, 1, 0, 1)$, giá trị tối ưu $f^* = 15$.



Hình 4.4. Lời giải bài toán cái túi theo thuật toán nhánh cận.

Chương trình giải bài toán cái túi theo thuật toán nhánh cận được thể hiện như sau:

```

#include <iostream.h>
#include <stdio.h>
#include <conio.h>
#define MAX 100
int A[MAX], C[MAX], F[MAX][MAX];
int XOPT[MAX], X[MAX];
int n, b, ind;
float W, FOPT=-32000, cost, weight=0;

FILE *fp;
  
```

```

void Init(void){
    fp = fopen("caituil.in","r");
    fscanf(fp,"%d%d", &n, &b);
    cout<<"\n So luong do vat:"<<n;
    cout<<"\n Trong luong tui:"<<b;
    for( int i=1; i<=n; i++)
        fscanf(fp,"%d%d", &A[i],&C[i]);
    cout<<"\n Vector trong luong:";
    for( i=1; i<=n; i++)
        cout<<A[i]<<" ";
    cout<<"\n Vector gia tri su dung:";
    for( i=1; i<=n; i++)
        cout<<C[i]<<" ";
    fclose(fp);
}
void Update_Kyluc(void){
    if (cost>FOPT) { FOPT =cost;
        for(int i=1; i<=n; i++)
            XOPT[i] = X[i];
    }
}
void Result(void) {
    cout<<"\n Ket qua toi uu:"<<FOPT;
    cout<<"\n Phuong an toi uu:";
    for(int i=1; i<=n; i++)
        cout<<XOPT[i]<<" ";
}
void Branch_And_Bound( int i) {
    int j, t =(b-weight)/A[i];
    for(j=t; j>=0; j--){ X[i] = j;
        weight = weight+A[i]*X[i];
        cost = cost + C[i]*X[i];
        if (i==n) Update_Kyluc();
        else if ( cost+C[i+1]*(b-weight)/A[i+1]>FOPT)
            Branch_And_Bound(i+1);
        weight = weight-A[i]*X[i];
        cost = cost - C[i]*X[i];
    }
}
void main(void) {
    Init();
    Branch_And_Bound(1);
    Result();
}

```

Ví dụ 2. Bài toán Người du lịch. Một người du lịch muốn đi thăm quan n thành phố $T_1, T_2, \dots, T_n$. Xuất phát từ một thành phố nào đó, người du lịch muốn đi qua tất cả các thành phố còn lại, mỗi thành phố đi qua đúng một lần, rồi quay trở lại thành phố xuất phát. Biết c_{ij} là chi phí đi từ thành phố T_i đến thành phố T_j ($i = 1, 2, \dots, n$), hãy tìm hành trình với tổng chi phí là nhỏ nhất (một hành trình là một cách đi thoả mãn điều kiện).

Giải. Cố định thành phố xuất phát là T_1 . Bài toán Người du lịch được đưa về bài toán: Tìm cực tiểu của phiếm hàm:

$$f(x_1, x_2, \dots, x_n) = c[1, x_2] + c[x_2, x_3] + \dots + c[x_{n-1}, x_n] + c[x_n, x_1] \rightarrow \min$$

với điều kiện

$$c_{\min} = \min\{c[i, j], i, j = 1, 2, \dots, n; i \neq j\} \text{ là chi phí đi lại giữa các thành phố.}$$

Giả sử ta đang có phương án bộ phận $(u_1, u_2, \dots, u_k)$. Phương án tương ứng với hành trình bộ phận qua k thành phố:

$$T_1 \rightarrow T(u_2) \rightarrow \dots \rightarrow T(u_{k-1}) \rightarrow T(u_k)$$

Vì vậy, chi phí phải trả theo hành trình bộ phận này sẽ là tổng các chi phí theo từng node của hành trình bộ phận.

$$\partial = c[1, u_2] + c[u_2, u_3] + \dots + c[u_{k-1}, u_k] .$$

Để phát triển hành trình bộ phận này thành hành trình đầy đủ, ta còn phải đi qua $n-k$ thành phố còn lại rồi quay trở về thành phố T_1 , tức là còn phải đi qua $n-k+1$ đoạn đường nữa. Do chi phí phải trả cho việc đi qua mỗi trong $n-k+1$ đoạn đường còn lại đều không nhiều hơn $c_{\min}$, nên cận dưới cho phương án bộ phận $(u_1, u_2, \dots, u_k)$ có thể được tính theo công thức

$$g(u_1, u_2, \dots, u_k) = \partial + (n - k + 1) c_{\min}.$$

Chẳng hạn ta giải bài toán người du lịch với ma trận chi phí như sau

$$C = \begin{vmatrix} 0 & 3 & 14 & 18 & 15 \\ 3 & 0 & 4 & 22 & 20 \\ 17 & 9 & 0 & 16 & 4 \\ 6 & 2 & 7 & 0 & 12 \\ 9 & 15 & 11 & 5 & 0 \end{vmatrix}$$

Ta có $c_{\min} = 2$. Quá trình thực hiện thuật toán được mô tả bởi cây tìm kiếm lời giải được thể hiện trong hình 4.2.

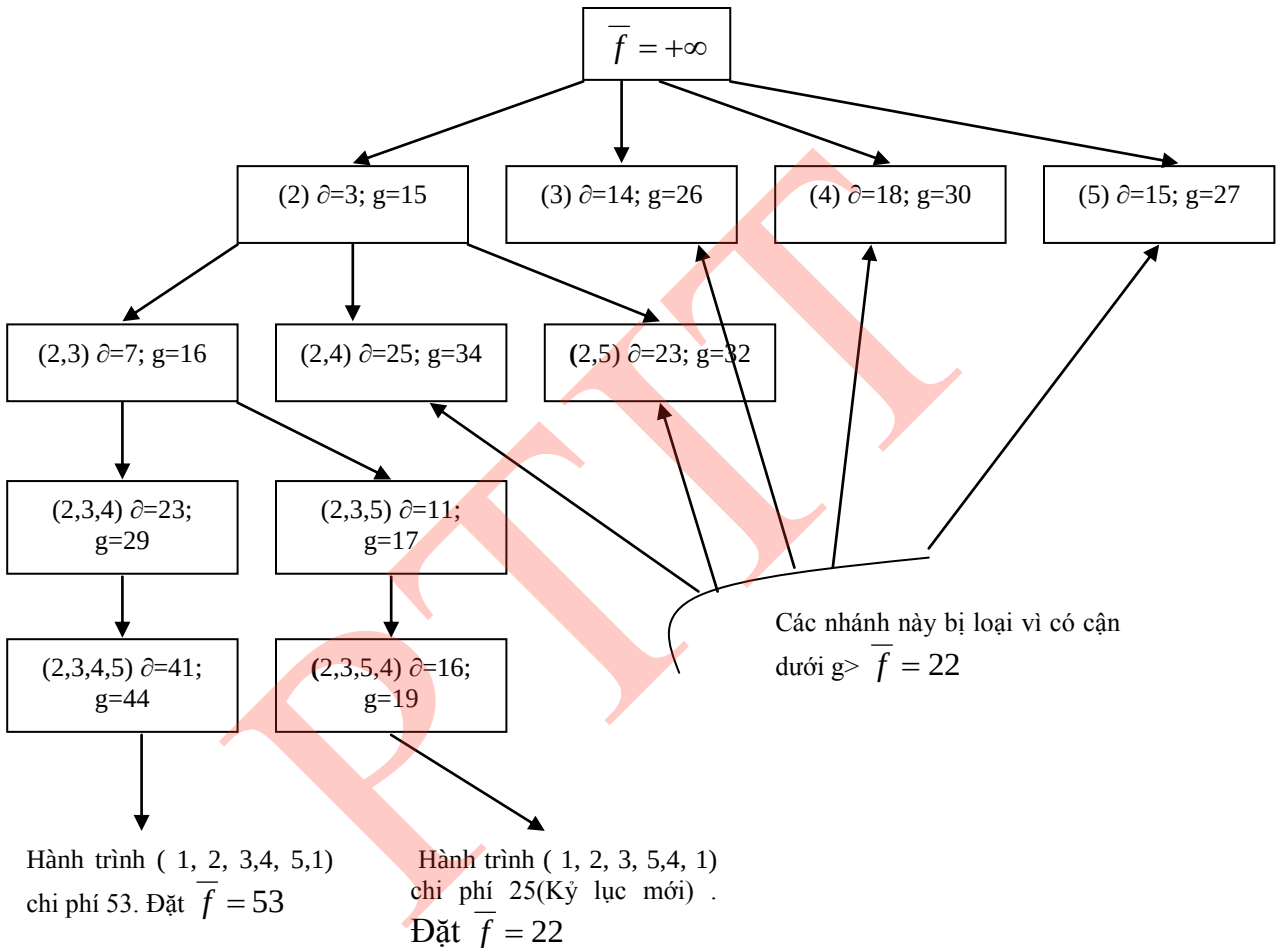
Thông tin về một phương án bộ phận trên cây được ghi trong các ô trên hình vẽ tương ứng theo thứ tự sau:

- đầu tiên là các thành phần của phương án.

- tiếp đến ∂ là chi phí theo hành trình bộ phận.
- g là cận dưới.

Kết thúc thuật toán, ta thu được phương án tối ưu (1, 2, 3, 5, 4, 1) tương ứng với phương án tối ưu với hành trình

$T_1 \rightarrow T_2 \rightarrow T_3 \rightarrow T_5 \rightarrow T_4 \rightarrow T_1$ và chi phí nhỏ nhất là 22 .



Hình 4.5. Cây tìm kiếm lời giải bài toán người du lịch.

Chương trình giải bài toán theo thuật toán nhánh cận được thể hiện như sau:

```
#include <stdio.h>
#include <stdlib.h>
#include <conio.h>
#include <io.h>
#define MAX 20
int n, P[MAX], B[MAX], C[20][20], count=0;
int A[MAX], XOPT[MAX];
```

```

int can, cmin, fopt;
void Read_Data(void){
    int i, j; FILE *fp;
    fp = fopen("dulich.in", "r");
    fscanf(fp, "%d", &n);
    printf("\n So thanh pho: %d", n);
    printf("\n Ma tran chi phi:");
    for (i=1; i<=n; i++){
        printf("\n");
        for(j=1; j<=n; j++){
            fscanf(fp, "%d", &C[i][j]);
            printf("%5d", C[i][j]);
        }
    }
}

int Min_Matrix(void){
    int min=1000, i, j;
    for(i=1; i<=n; i++){
        for(j=1; j<=n; j++){
            if (i!=j && min>C[i][j])
                min=C[i][j];
        }
    }
    return(min);
}

void Init(void){
    int i;
    cmin=Min_Matrix();
    fopt=32000; can=0; A[1]=1;
    for (i=1; i<=n; i++)
        B[i]=1;
}

void Result(void){
    int i;
    printf("\n Hanh trinh toi uu %d:", fopt);
    printf("\n Hanh trinh:");
    for(i=1; i<=n; i++)
        printf("%3d->", XOPT[i]);
    printf("%d", 1);
}

void Swap(void){
    int i;
    for(i=1; i<=n; i++)

```

```

        XOPT[i]=A[i];
    }
    void Update_Kyluc(void){
        int sum;
        sum=can+C[A[n]][A[1]];
        if(sum<fopt) {
            Swap();
            fopt=sum;
        }
    }
    void Try(int i){
        int j;
        for(j=2; j<=n;j++){
            if(B[j]){
                A[i]=j; B[j]=0;
                can=can+C[A[i-1]][A[i]];
                if (i==n) Update_Kyluc();
                else if( can + (n-i+1)*cmin< fopt){
                    count++;
                    Try(i+1);
                }
                B[j]=1;can=can-C[A[i-1]][A[i]];
            }
        }
    }
}
void main(void){
    clrscr();Read_Data();Init();
    Try(2);Result();
    getch();
}

```

4.4. Kỹ thuật rút gọn giải quyết bài toán người du lịch

Thuật toán nhánh cận là phương pháp chủ yếu để giải các bài toán tối ưu tổ hợp. Tư tưởng cơ bản của thuật toán là trong quá trình tìm kiếm lời giải, ta sẽ phân hoạch tập các phương án của bài toán thành hai hay nhiều tập con biểu diễn như một node của cây tìm kiếm và cố gắng bằng phép đánh giá cận các node, tìm cách loại bỏ những nhánh cây (những tập con các phương án của bài toán) mà ta biết chắc chắn không phương án tối ưu. Mặc dù trong trường hợp tồi nhất thuật toán sẽ trở thành duyệt toàn bộ, nhưng trong những trường hợp cụ thể nó có thể rút ngắn đáng kể thời gian tìm kiếm. Mục này sẽ thể hiện khác những tư tưởng của thuật toán nhánh cận vào việc giải quyết bài toán người du lịch.

Xét bài toán người du lịch như đã được phát biểu. Gọi $C = \{ c_{ij}: i, j = 1, 2, \dots, n \}$ là ma trận chi phí. Mỗi hành trình của người du lịch

$T_{\pi(1)} \rightarrow T_{\pi(2)} \rightarrow \dots \rightarrow T_{\pi(n)} \rightarrow T_{\pi(1)}$ có thể viết lại dưới dạng

$(\pi(1), \pi(2), \pi(2), \pi(3), \dots, \pi(n-1), \pi(n), \pi(n), \pi(1))$, trong đó mỗi thành phần

$\pi(j-1), \pi(j)$ sẽ được gọi là một cạnh của hành trình.

Khi tiến hành tìm kiếm lời giải bài toán người du lịch chúng ta phân tập các hành trình thành 2 tập con: Tập những hành trình chứa một cặp cạnh (i, j) nào đó còn tập kia gồm những hành trình không chứa cạnh này. Ta gọi việc làm đó là sự phân nhánh, mỗi tập con như vậy được gọi là một nhánh hay một node của cây tìm kiếm. Quá trình phân nhánh được minh họa bởi cây tìm kiếm như trong Hình 4.6.



(Hình 4.6)

Việc phân nhánh sẽ được thực hiện dựa trên một qui tắc heuristic nào đó cho phép ta rút ngắn quá trình tìm kiếm phương án tối ưu. Sau khi phân nhánh và tính cận dưới giá trị hàm mục tiêu trên mỗi tập con. Việc tìm kiếm sẽ tiếp tục trên tập con có giá trị cận dưới nhỏ hơn. Thủ tục này được tiếp tục cho đến khi ta nhận được một hành trình đầy đủ tức là một phương án của bài toán. Khi đó ta chỉ cần xét những tập con các phương án nào có cận dưới nhỏ hơn giá trị của hàm mục tiêu tại phương án đã tìm được. Quá trình phân nhánh và tính cận trên tập các phương án của bài toán thông thường cho phép rút ngắn một cách đáng kể quá trình tìm kiếm do ta loại được rất nhiều tập con chắc chắn không chứa phương án tối ưu. Sau đây, là một kỹ thuật nữa của thuật toán.

4.4.1. Thủ tục rút gọn

Rõ ràng tổng chi phí của một hành trình của người du lịch sẽ chứa đúng một phần tử của mỗi dòng và đúng một phần tử của mỗi cột trong ma trận chi phí C . Do đó, nếu ta cộng hay trừ bớt mỗi phần tử của một dòng (hay cột) của ma trận C đi cùng một số α thì độ dài của tất cả các hành trình đều giảm đi α vì thế hành trình tối ưu cũng sẽ không bị thay đổi. Vì vậy, nếu ta tiến hành bớt đi các phần tử của mỗi dòng và mỗi cột đi một hằng số sao cho ta thu được một ma trận gồm các phần tử không âm mà trên mỗi dòng, mỗi cột đều có ít nhất một số 0, thì tổng các số trừ đó cho ta cận dưới của mọi hành trình. Thủ tục bớt này được gọi là thủ tục rút gọn, các hằng số trừ ở mỗi dòng (cột) sẽ được gọi là hằng

số rút gọn theo dòng(cột), ma trận thu được được gọi là ma trận rút gọn. Thủ tục sau cho phép rút gọn ma trận một ma trận A kích thước $k \times k$ đồng thời tính tổng các hằng số rút gọn.

```
float Reduce( float A[][max], int k) {
    sum =0;
    for (i = 1; i ≤ k; i++){
        r[i] = <phần tử nhỏ nhất của dòng i >;
        if (r[i] > 0 ) {
            <Bớt mỗi phần tử của dòng i đi r[i] >;
            sum = sum + r[i];
        }
    }
    for (j=1; j ≤ k; j++) {
        s[j] := <Phần tử nhỏ nhất của cột j>;
        if (s[j] > 0 )
            sum = sum + S[j];
    }
    return(sum);
}
```

Ví dụ. Giả sử ta có ma trận chi phí với $n=6$ thành phố sau:

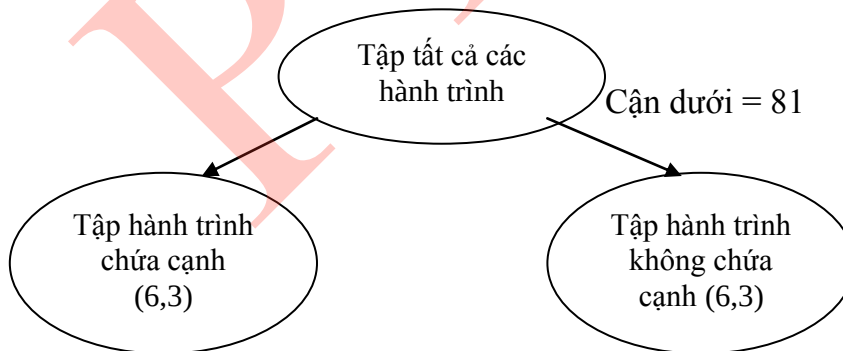
	1	2	3	4	5	6	r[i]
1	∞	3	93	13	33	9	3
2	4	∞	77	42	21	16	4
3	45	17	∞	36	16	28	16
4	39	90	80	∞	56	7	7
5	28	46	88	33	∞	25	25
6	3	88	18	46	92	∞	3
	0	0	15	8	0	0	

Đầu tiên trừ bớt mỗi phần tử của các dòng 1, 2, 3, 4, 5, 6 cho các hằng số rút gọn tương ứng là (3, 4, 16, 7, 25, 3) , sau đó trong ma trận thu được ta tìm được phần tử nhỏ khác 0 của cột 3 và 4 tương ứng là (15, 8). Thực hiện rút gọn theo cột ta nhận được ma trận sau:

	1	2	3	4	5	6
1	∞	0	75	2	30	6
2	0	∞	58	30	17	12
3	29	1	∞	12	0	12
4	32	83	58	∞	49	0
5	3	21	48	0	∞	0
6	0	85	0	35	89	∞

Tổng các hằng số rút gọn là 81, vì vậy cận dưới cho tất cả các hành trình là 81 (không thể có hành trình có chi phí nhỏ hơn 81).

Bây giờ ta xét cách phân tập các phương án ra thành hai tập. Giả sử ta chọn cạnh (6, 3) để phân nhánh. Khi đó tập các hành trình được phân thành hai tập con, một tập là các hành trình chứa cạnh (6,3), còn tập kia là các hành trình không chứa cạnh (6,3). Vì biết cạnh (6, 3) không tham gia vào hành trình nên ta cấm hành trình đi qua cạnh này bằng cách đặt $C[6, 3] = \infty$. Ma trận thu được sẽ có thể rút gọn bằng cách bớt đi mỗi phần tử của cột 3 đi 48 (hàng 6 giữ nguyên). Như vậy ta thu được cận dưới của hành trình không chứa cạnh (6,3) là $81 + 48 = 129$. Còn đối với tập chứa cạnh (6, 3) ta phải loại dòng 6, cột 3 khỏi ma trận tương ứng với nó, bởi vì đã đi theo cạnh (6, 3) thì không thể đi từ 6 sang bất sang bất cứ nơi nào khác và cũng không được phép đi bất cứ đâu từ 3. Kết quả nhận được là ma trận với bậc giảm đi 1. Ngoài ra, do đã đi theo cạnh (6, 3) nên không được phép đi từ 3 đến 6 nữa, vì vậy cần cấm đi theo cạnh (3, 6) bằng cách đặt $C(3, 6) = \infty$. Cây tìm kiếm lúc này có dạng như trong hình 4.7.



(Hình 4.7)

Cận dưới = 81

Cận dưới = 129

	1	2	4	5	6		1	2	3	4	5	6
1	∞	0	2	30	6	1	∞	0	27	2	30	6
2	0	∞	30	17	12	2	0	∞	10	30	17	12
3	29	1	12	0	∞	3	29	1	∞	12	0	12
4	32	83	∞	49	0	4	32	83	10	∞	49	0
5	3	21	0	∞	0	5	3	21	0	0	∞	0
						6	0	85	∞	35	89	∞

Cạnh (6,3) được chọn để phân nhánh vì phân nhánh theo nó ta thu được cận dưới của nhánh bên phải là lớn nhất so với việc phân nhánh theo các cạnh khác. Qui tắc này sẽ được áp dụng ở để phân nhánh ở mỗi đỉnh của cây tìm kiếm. Trong quá trình tìm kiếm chúng ta luôn đi theo nhánh bên trái trước. Nhánh bên trái sẽ có ma trận rút gọn với bậc giảm đi 1. Trong ma trận của nhánh bên phải ta thay một số bởi ∞ , và có thể rút gọn thêm được ma trận này khi tính lại các hằng số rút gọn theo dòng và cột tương ứng với cạnh phân nhánh, nhưng kích thước của ma trận vẫn giữ nguyên.

Do cạnh chọn để phân nhánh phải là cạnh làm tăng cận dưới của nhánh bên phải lên nhiều nhất, nên để tìm nó ta sẽ chọn số không nào trong ma trận mà khi thay nó bởi ∞ sẽ cho ta tổng hằng số rút gọn theo dòng và cột chứa nó là lớn nhất. Thủ tục đó có thể được mô tả như sau để chọn cạnh phân nhánh (r, c).

4.4.2. Thủ tục chọn cạnh phân nhánh (r,c)

void BestEdge(A, k, r, c, beta)

Đầu vào: Ma trận rút gọn A kích thước $k \times k$

Kết quả ra: Cạnh phân nhánh (r,c) và tổng hằng số rút gọn theo dòng r cột c là beta.

```
{
    beta =  $-\infty$ ;
    for ( i = 1; i  $\leq$  k; i++){
        for ( j = 1; j  $\leq$  k; j++){
            if (A[i,j] == 0){
                minr = <phần tử nhỏ nhất trên dòng i khác với A[i,j];
                minc = <phần tử nhỏ nhất trên cột j khác với A[i,j];
                total = minr + minc;
```

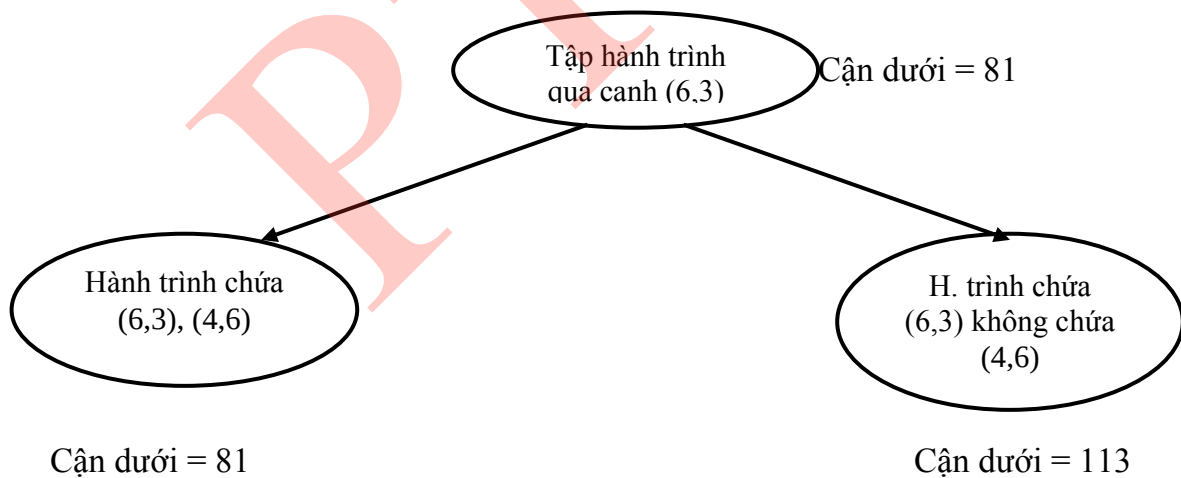
```

if (total > beta ) {
    beta = total;
    r = i; /* Chỉ số dòng tốt nhất*/
    c = j; /* Chỉ số cột tốt nhất*/
}
}
}
}
}
}

```

Trong ma trận rút gọn 5×5 của nhánh bên trái hình 5.4, số không ở vị trí (4, 6) sẽ cho tổng hàng số rút gọn là 32 (theo dòng 4 là 32, cột 6 là 0). Đây là hệ số rút gọn có giá trị lớn nhất đối với các số không của ma trận này. Việc phân nhánh tiếp tục sẽ dựa vào cạnh (4, 6). Khi đó cạnh dưới của nhánh bên phải tương ứng với tập hành trình đi qua cạnh (6,3) nhưng không đi qua cạnh (4, 6) sẽ là $81 + 32 = 113$. Còn nhánh bên trái sẽ tương ứng với ma trận 4×4 (vì ta phải loại bỏ dòng 4 và cột 6). Tình huống phân nhánh này được mô tả trong Hình 4.7.

Nhận thấy rằng vì cạnh (4, 6) và (6, 3) đã nằm trong hành trình nên cạnh (3, 4) không thể đi qua được nữa (nếu đi qua ta sẽ có một hành trình con từ những thành phố này). Để ngăn cấm việc tạo thành các hành trình con ta sẽ gán cho phần tử ở vị trí (3, 4) giá trị ∞ .



(Hình 4.8)

Ngăn cấm tạo thành hành trình con:

Tổng quát hơn, khi phân nhánh dựa vào cạnh (i_u, i_v) ta phải thêm cạnh này vào danh sách các cạnh của node bên trái nhất. Nếu i_u là đỉnh cuối của một đường đi $(i_1, i_2, \dots, i_u)$ và j_v là đỉnh đầu của đường đi $(j_1, j_2, \dots, j_k)$ thì để ngăn ngừa khả năng tạo thành hành trình con ta phải ngăn ngừa khả năng tạo thành hành trình con ta phải cấm cạnh (j_k, i_1) . Để